

Method of Moments

Math 742

2026-01-30

Introduction

So far this semester, we have learned about sampling distributions, plug-in estimators and sufficient statistics. We will look at several methods of creating estimators. In this lecture, we will look at the **Method of Moments** technique for constructing an estimator. MoM provides a first way to construct estimators before we learn MLE.

The basic idea is that we will match sample moments to population moments. This method is pretty simple, intuitive and was the first widely used estimation method.

Imagine shining a light on a 3D object and all you can see is its shadow, not the object itself. The shadow is obviously not the object, but it captures enough information about its shape that we might be able to reproduce it. Maybe, we actually don't have one shadow but multiple shadows from different angles.

- One shadow shows the center of the object, this would be like the mean (first moment).
- One shadow shows how much space the object covers (i.e. its spread), this would be like the variance (i.e. 2nd moment).
- One shadow shows us whether the object is symmetrical (i.e. its skewness), this would be the 3rd moment.

Just like a 2D shadow of a 3D object:

- it preserves the rough shape,
- but loses details like texture, depth, contour, and shading,

Method of Moments works the same way:

- it preserves low-level summaries (mean, variance),
- but loses higher-order information about the full distribution (information captured by the likelihood).

In contrast, **Maximum Likelihood Estimation (which we will learn later)** uses all the information in the sample. MLE matches the entire shape of the distribution, while MoM typically matches only the first moment (or first few moments, not all moments).

So MoM is simple and intuitive, but it discards information that the likelihood keeps, which is why MoM estimators are not always as efficient as MLE estimators.

Definitions

We want to distinguish between the moments of the population and the moments of the sample.

$$\begin{aligned} \text{Population Moment: } \mu'_k &= \mathbb{E}(X^k) \\ \mu'_1 &= \mathbb{E}(X), \mu'_2 = \mathbb{E}(X^2), \mu'_3 = \mathbb{E}(X^3), \dots \end{aligned}$$

$$\text{Sample Moment: } m'_k = \frac{1}{n} \sum_{i=1}^n X_i^k$$

$$m'_1 = \frac{1}{n} \sum_{i=1}^n X_i, \quad m'_2 = \frac{1}{n} \sum_{i=1}^n X_i^2, \quad m'_3 = \frac{1}{n} \sum_{i=1}^n X_i^3, \dots$$

Method of Moments fundamental idea (assumption):

For any parameter θ we are trying to estimate,

$$m'_k \approx \mu'_k(\theta)$$

MoM works when there are as many moments as parameters, and when the resulting equations identify the parameter uniquely.

Examples

Bernouli(p)

The first sample moment $m'_1 = \hat{p} = \frac{1}{n} \sum_{i=1}^n X_i = \bar{X}$

The first population moment $\mu'_1 = \mathbb{E}(X^1) = \mathbb{E}(X) = p$

So, the Mom estimator of p is the sample proportion ($\hat{p}$).

Poisson(λ)

If $X \sim \text{Poisson}(\lambda)$, $\mathbb{E}(X) = \lambda$.

The sample first sample moment is, $m'_1 = \bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$

So, $\hat{\lambda} = \bar{X}$.

Exponential(λ)

Population mean: $\mathbb{E}(X) = \frac{1}{\lambda}$.

MoM: $\bar{X} = 1/\lambda \implies \hat{\lambda} = 1/\bar{X}$.

$N(\mu, \sigma^2)$

Here we have 2 moments,

$$\mu'_1 = \mathbb{E}(X) \text{ and } \mu'_2 = \mathbb{E}(X^2) = \mu^2 + \sigma^2$$

Note: MoM uses raw moments, not central moments. i.e. $\mu'_2 = \mathbb{E}(X^2)$ not $\mathbb{E}(X - \bar{X})^2$.

Now equate sample moments,

$$\bar{X} = \mu$$

$$\frac{1}{n} \sum X_i^2 = \mu^2 + \sigma^2$$

This gives us,

$$\hat{\mu} = \bar{X} \text{ and } \hat{\sigma}^2 = \frac{1}{n} \sum X_i^2 - \bar{X}^2$$

It should be pointed out that the MoM estimator of the variance is biased. Since we showed,

$$\mathbb{E}(S^2) = \sigma^2$$

$$S^2 = \frac{1}{n-1} \left(\sum X_i^2 - n\bar{X}^2 \right)$$

the sample variance is unbiased.

Strengths/Weaknesses

→ Strengths:

- Easy to compute
- Always exists when the corresponding moment exists
- Usually gives estimates that are reasonable
- Works for complicated distributions when MLE is difficult
- Always gives nice closed-form solutions

→ Weaknesses

- Not always efficient → *has larger variance than the Cramér–Rao bound or the MLE* (efficiency will be covered later)
- Can be biased
- Requires finite moments → doesn't work for heavy tailed distributions (e.g. Cauchy)
- Can behave poorly for small n

Simulation

Exponential Simulation

$$\hat{\lambda}_{\text{MoM}} = \frac{1}{\bar{X}}$$

```
library(ggplot2)

set.seed(314159)

# True parameter
lambda <- 2
n <- 30          # sample size
Nsim <- 5000      # number of simulated experiments

# Storage
mom <- numeric(Nsim)

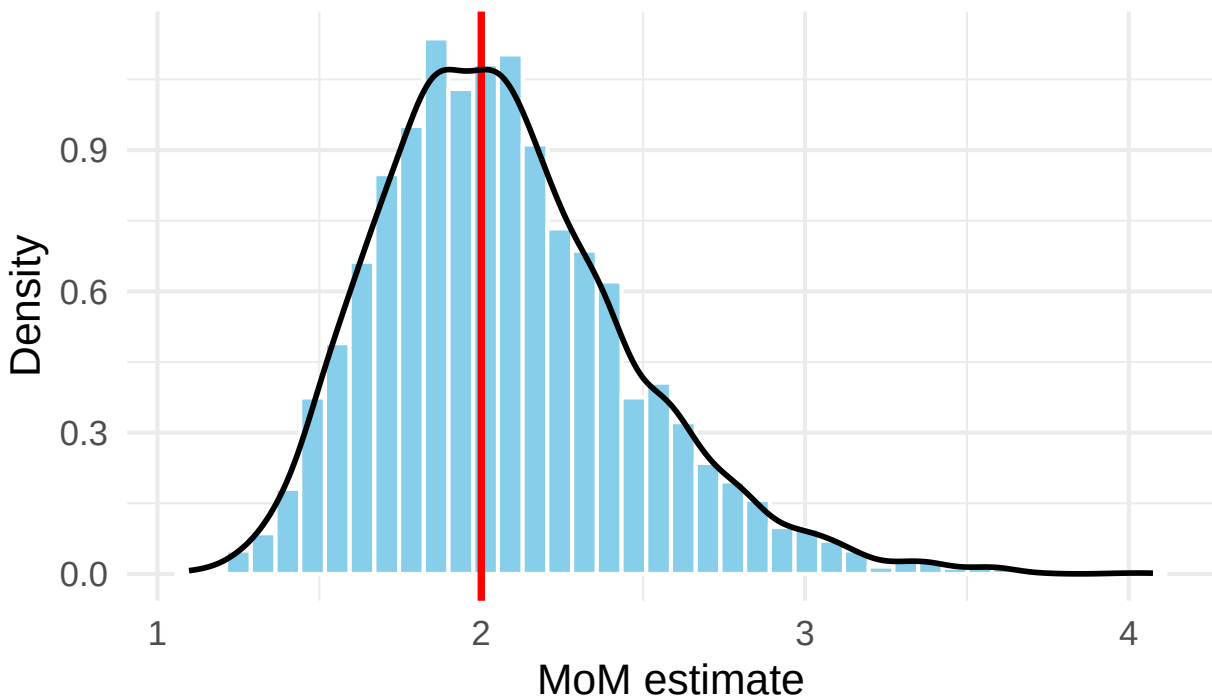
# Simulation loop
for (i in 1:Nsim) {
  x <- rexp(n, rate = lambda)
  mom[i] <- 1 / mean(x)      # Method of Moments estimate
}
```

```
# Plot sampling distribution
df <- data.frame(mom = mom)

ggplot(df, aes(mom)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40, fill = "skyblue", color = "white") +
  geom_vline(xintercept = lambda, color = "red", size = 1.3) +
  labs(title = "Method of Moments for Exponential(lambda)",
        subtitle = "Red line = true lambda",
        x = "MoM estimate", y = "Density") +
  geom_density(size = 1) +
  theme_minimal(base_size = 16)
```

Method of Moments for Exponential(lambda)

Red line = true lambda



Bernoulli(p)

$$\hat{p}_{\text{MoM}} = \bar{X}$$

```
library(ggplot2)

set.seed(999)

p <- 0.3
n <- 200
Nsim <- 5000

mom <- numeric(Nsim)
```

```

for (i in 1:Nsim) {
  x <- rbinom(n, 1, p)
  mom[i] <- mean(x) # MoM estimator
}

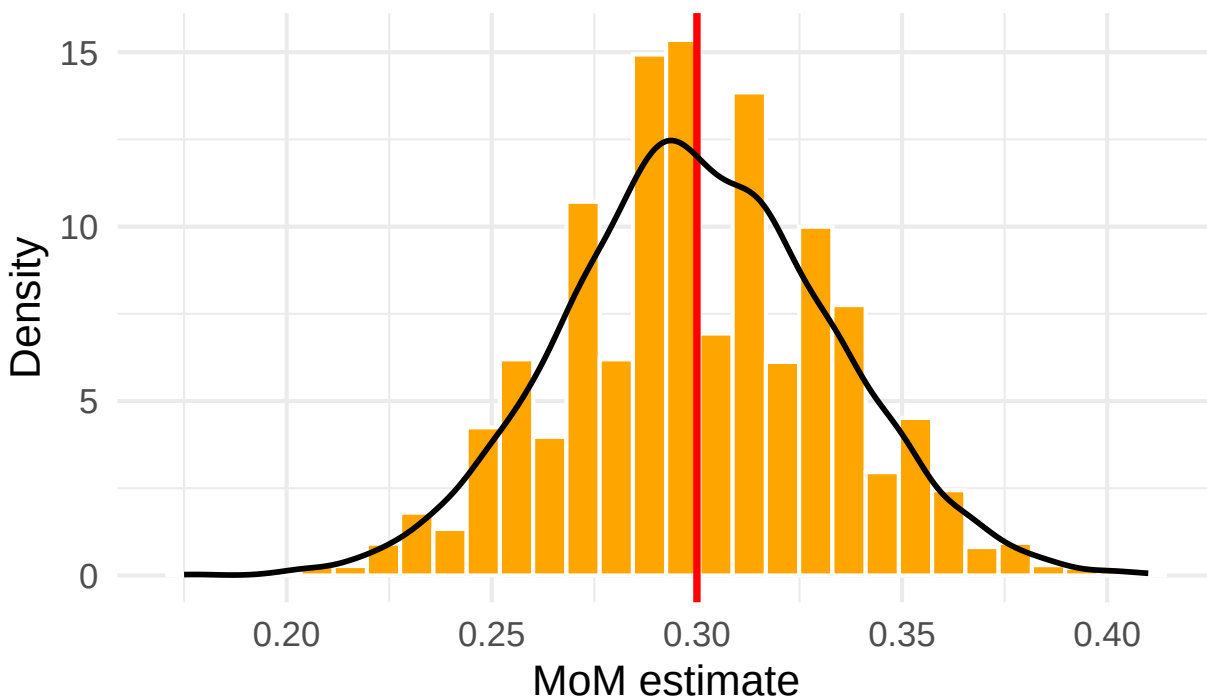
df <- data.frame(mom = mom)

ggplot(df, aes(mom)) +
  geom_histogram(aes(y =after_stat(density)), bins = 30, fill = "orange", color = "white") +
  geom_vline(xintercept = p, color = "red", linewidth = 1.3) +
  labs(title = "Method of Moments for Bernoulli(p)",
       subtitle = "Red line = true p",
       x = "MoM estimate", y = "Density") +
  geom_density(size = 1)+
  theme_minimal(base_size = 16)

```

Method of Moments for Bernoulli(p)

Red line = true p



$N(\mu, \sigma^2)$

$$\hat{\mu}_{\text{MoM}} = \bar{X}$$

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n X_i^2 - \bar{X}^2$$

```

library(ggplot2)
library(patchwork)

set.seed(671965)

mu <- 5
sigma <- 2
n <- 40
Nsim <- 5000

mom_mu <- numeric(Nsim)
mom_sigma2 <- numeric(Nsim)

for (i in 1:Nsim) {
  x <- rnorm(n, mu, sigma)
  mom_mu[i] <- mean(x)
  mom_sigma2[i] <- mean(x^2) - mean(x)^2  # MoM estimator of variance
}

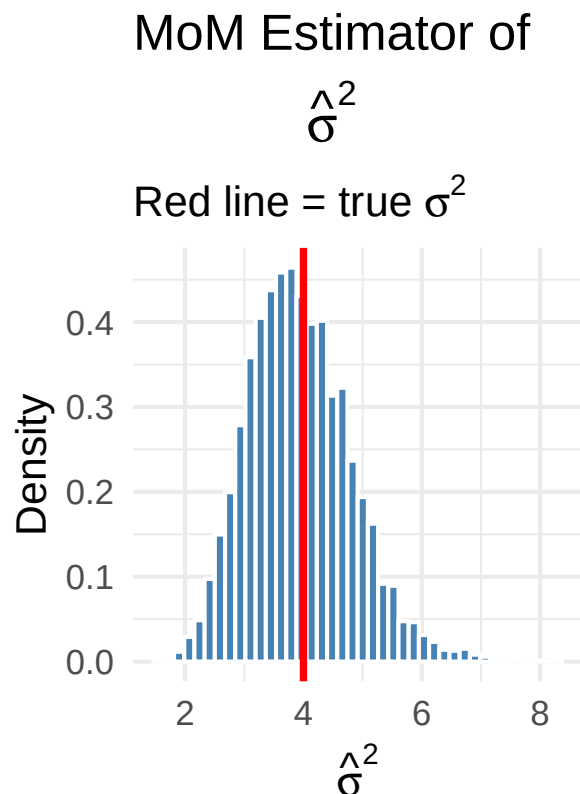
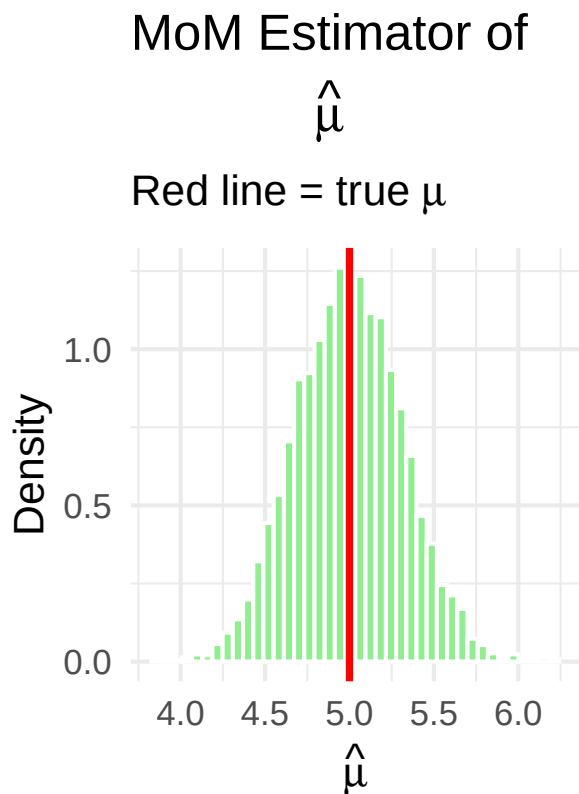
df <- data.frame(mom_mu = mom_mu, mom_sigma2 = mom_sigma2)

# Plot mu estimates
p1 <- ggplot(df, aes(mom_mu)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40, fill = "lightgreen", color = "white") +
  geom_vline(xintercept = mu, color = "red", linewidth = 1.3) +
  labs(
    title = expression(atop(
      "MoM Estimator of",
      hat(mu)
    )),
    subtitle = expression("Red line = true " * mu),
    x = expression(hat(mu)),
    y = "Density"
  ) +
  theme_minimal(base_size = 16)

# Plot sigma^2 estimates
p2 <- ggplot(df, aes(mom_sigma2)) +
  geom_histogram(aes(y = after_stat(density)), bins = 40, fill = "steelblue", color = "white") +
  geom_vline(xintercept = sigma^2, color = "red", linewidth = 1.3) +
  labs(
    title = expression(atop(
      "MoM Estimator of",
      hat(sigma)^2
    )),
    subtitle = expression("Red line = true " * sigma^2),
    x = expression(hat(sigma)^2),
    y = "Density"
  ) +
  theme_minimal(base_size = 16)

p1 | p2

```



Summary

- Method of Moments (MoM) = match sample moments to population moments.
- Very intuitive: “match what the sample looks like to what the model says.”
- Usually simple, often fast.
- Next we will cover maximum likelihood estimation (MLE). Hopefully, understanding MoM will give you intuition/insight into MLE.
- MLE typically uses more information than MoM, so MoM provides a simple baseline for intuition before we learn likelihood-based estimation.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(314159)

# True parameter and simulation settings
lambda_true = 2.0
n = 30
Nsim = 5000

# MoM: lambda_hat = 1 / Xbar, with X ~ Exp(rate=lambda_true)
mom = np.array([
```

```

1.0 / np.mean(np.random.exponential(scale=1.0/lambda_true, size=n))
for _ in range(Nsim)
])

def density_curve(samples, xgrid):
    try:
        from scipy.stats import gaussian_kde
        kde = gaussian_kde(samples)
        return kde(xgrid)
    except Exception:
        # fallback: normal approximation
        m = samples.mean()
        s = samples.std(ddof=1)
        return (1/(s*np.sqrt(2*np.pi))) * np.exp(-(xgrid-m)**2/(2*s**2))

fig, ax = plt.subplots(figsize=(8, 5))

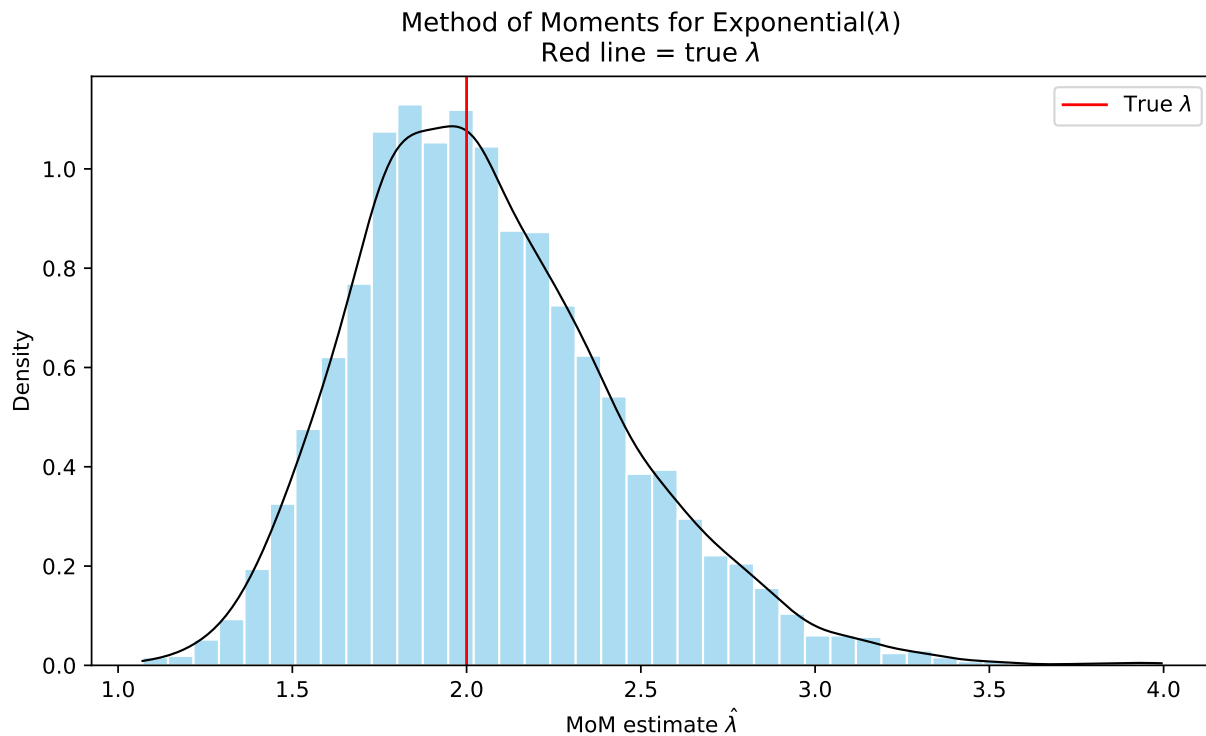
_ = ax.hist(mom, bins=40, density=True,
            color="skyblue", edgecolor="white", alpha=0.7)

xgrid = np.linspace(mom.min(), mom.max(), 500)
ax.plot(xgrid, density_curve(mom, xgrid), color="black", linewidth=1)

ax.axvline(lambda_true, color="red", linewidth=1.3, label=r"True  $\lambda$ ")
ax.set_title(r"Method of Moments for Exponential( $\lambda$ ) " "\n" r"Red line = true  $\lambda$ ")
ax.set_xlabel(r"MoM estimate  $\hat{\lambda}$ ")
ax.set_ylabel("Density")
ax.legend()

fig.tight_layout()
plt.show()

```

```
import numpy as np
import matplotlib.pyplot as plt

# -----
# Simulation
# -----
np.random.seed(999)

p = 0.3
n = 200
Nsim = 5000

mom = np.array([
    np.mean(np.random.binomial(1, p, size=n))
    for _ in range(Nsim)
])

# -----
# KDE (true density, ggplot-like)
# -----
from scipy.stats import gaussian_kde
kde = gaussian_kde(mom)

xgrid = np.linspace(mom.min(), mom.max(), 500)

# -----
# Plot
# -----
fig, ax = plt.subplots(figsize=(8, 5))
```

```

# Histogram (density)
_ = ax.hist(
    mom,
    bins=30,
    density=True,
    color="orange",
    edgecolor="white"
)

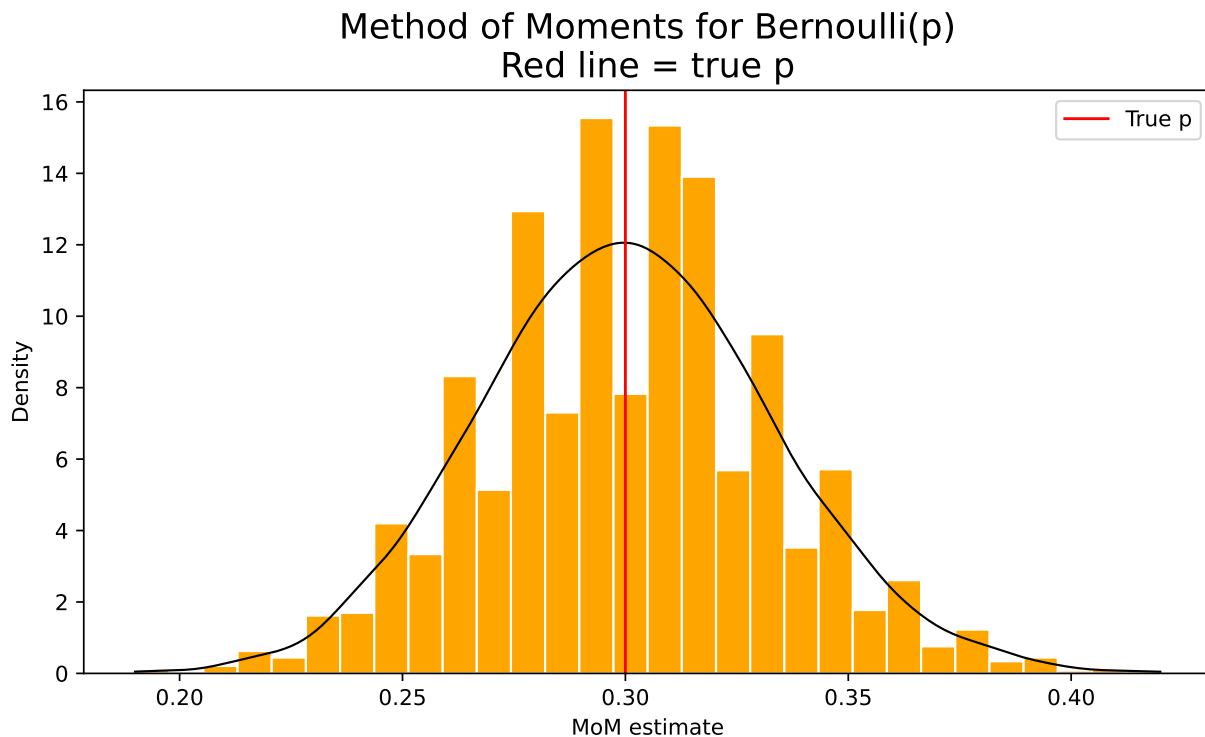
# Density curve (geom_density equivalent)
ax.plot(xgrid, kde(xgrid), color="black", linewidth=1)

# True p line
ax.axvline(p, color="red", linewidth=1.3, label="True p")

# Titles and labels
ax.set_title("Method of Moments for Bernoulli(p)\nRed line = true p", fontsize=16)
ax.set_xlabel("MoM estimate")
ax.set_ylabel("Density")

ax.legend()
fig.tight_layout()
plt.show()

```



```

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(671965)

```

```

mu_true = 5.0
sigma_true = 2.0
sigma2_true = sigma_true**2
n = 40
Nsim = 5000

mom_mu = np.empty(Nsim)
mom_sigma2 = np.empty(Nsim)

for i in range(Nsim):
    x = np.random.normal(mu_true, sigma_true, size=n)
    m = x.mean()
    mom_mu[i] = m
    mom_sigma2[i] = (x**2).mean() - m**2 # MoM variance (ddof=0 style)

def density_curve(samples, xgrid):
    try:
        from scipy.stats import gaussian_kde
        kde = gaussian_kde(samples)
        return kde(xgrid)
    except Exception:
        m = samples.mean()
        s = samples.std(ddof=1)
        return (1/(s*np.sqrt(2*np.pi))) * np.exp(-(xgrid-m)**2/(2*s**2))

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# --- mu ---
_ = axes[0].hist(mom_mu, bins=40, density=True,
                 color="lightgreen", edgecolor="white", alpha=0.7)
x0 = np.linspace(mom_mu.min(), mom_mu.max(), 500)
axes[0].plot(x0, density_curve(mom_mu, x0), color="black", linewidth=1)
axes[0].axvline(mu_true, color="red", linewidth=1.3, label=r"True  $\mu$ ")
axes[0].set_title(r"MoM Estimator of  $\mu$ " "\n" r"Red line = true  $\mu$ ")
axes[0].set_xlabel(r"Estimate  $\hat{\mu}$ ")
axes[0].set_ylabel("Density")
axes[0].legend()

# --- sigma^2 ---
_ = axes[1].hist(mom_sigma2, bins=40, density=True,
                 color="steelblue", edgecolor="white", alpha=0.7)
x1 = np.linspace(mom_sigma2.min(), mom_sigma2.max(), 500)
axes[1].plot(x1, density_curve(mom_sigma2, x1), color="black", linewidth=1)
axes[1].axvline(sigma2_true, color="red", linewidth=1.3, label=r"True  $\sigma^2$ ")
axes[1].set_title(r"MoM Estimator of  $\sigma^2$ " "\n" r"Red line = true  $\sigma^2$ ")
axes[1].set_xlabel(r"Estimate  $\widehat{\sigma^2}$ ")
axes[1].set_ylabel("Density")
axes[1].legend()

fig.tight_layout()
plt.show()

```

